AB Power Meter Monitor

Release 1.7.0 — Technical Manual

Allen-Bradley 1403 site power-meter poller, data logger, GUI and Prometheus exporter for NRAO / Green Bank Observatory

Component	Version
`ab_power_meter_monitor.py`	1.7.0
`ab_live_plots.py` (GUI plots)	0.1.0
`vszh5_writer.py`	0.0.19
`ab_prometheus_exporter.py`	0.2.0
`ab_grafana_dashboard.py`	0.0.1
`ab_meter_caller.py`	1.3.0
Python (primary, tested)	3.12.3
Python 3.7 (pinned, tested)	3.7.12
pyqtgraph (3.12 / 3.8 / 3.7)	0.14.0 / 0.13.3 / 0.12.4
Windows Python (Windows 11 and Wine 9.0)	3.12.15
Veusz (viewer, verified)	4.2.1
Grafana / Prometheus (verified)	12.2 / 3.5

William W. Wallace
NRAO / Green Bank Observatory
wwallace@nrao.edu

27 October 2026

Contents

1 Summary **2**

2 Findings: RAM flush **2**
2.1 Why the flush did not protect the GUI . . . 2
2.2 What 1.6.0 does . . . 3
2.3 Store estimate and trigger . . . 3
2.4 Returning memory to the operating system (1.6.1) . . . 3

3 Findings: iterative .vszh5 writing **4**
3.1 How 1.5.5 wrote Veusz files . . . 4
3.2 How 1.6.0 writes them . . . 5

4 Other defects fixed **5**

5 Architecture **6**
5.1 Threads . . . 6
5.2 Persistence by sequence cursors . . . 6

6 Windows operation (1.6.2, 1.6.3) **7**
6.1 Output files held open by another program . . . 7
6.2 Stopping with Ctrl+Break . . . 7
6.3 Windows test kit . . . 8
6.4 First results on real Windows (1.6.3) . . . 8

7 Prometheus exporter 0.2.0 **9**

8 Prometheus and Grafana (1.6.4) **9**
8.1 Two ways to run the exporter . . . 9
8.2 Dashboard . . . 9
8.3 Python 3.7 on RHEL 9 . . . 11
8.4 Verification . . . 11

9 GUI **11**
9.1 Interactive plots (1.7.0) . . . 11

10 Deployment **13**
10.1 Environment (mamba) . . . 13
10.2 Services . . . 13
10.3 Library use . . . 13

11 Configuration reference **13**

12 Verification **14**

13 Module review **16**

14 Known limitations and recommendations **17**

A Distribution contents **20**

1 Summary

Release 1.6.0 makes the memory protection of the monitor work in the GUI and in headless operation, replaces the Veusz output with native `.vszh5` documents that are appended in place on every poll, fixes a FITS defect that multiplied rows on every rewrite, and corrects the Prometheus exporter so it never re-exports stale readings. Python 3.12 is the primary, tested target; the source remains at the Python 3.7 language level. Every defect below was first reproduced against an offline emulator of the meter web pages and then verified fixed (Section 12).

Release 1.6.1 makes the flush hand the freed memory back to *Windows* as well (1.6.0 did so only on Linux), Section 2.4, and fixes a race in which a forced write (flush, Save Now, stop) waiting for a busy writer could send the same rows twice (Section 4). The race was found by running the test-suite on a Windows build of Python 3.12.

Release 1.6.2 makes Windows operation robust: an output file held open by another program (Excel) no longer produces an error per table per poll, and its rows are kept or written to rescue files that are merged back; Ctrl+Break now stops the headless monitor and the exporter gracefully. A Windows test kit (Section 6) runs every check on a Windows PC.

Release 1.6.3 follows the first run of that kit on real Windows hardware (Section 6.4): the GUI crash seen there was in Intel oneMKL 2026.1.0, NumPy's matrix library in the conda-forge environment, and is avoided with OpenBLAS; NumPy's matrix library is limited to one thread (2.97 GB → 18 MB commit charge); an unreachable meter no longer costs a minute per poll; and a run without any data says so.

Release 1.6.4 adds live dashboards (Section 8): exporter 0.2.0 runs continuously or as single runs pushed to a Pushgateway, returns the meters' dict after every poll and writes no files; a builder makes a Grafana dashboard with one row per meter and one plot per unit. Python 3.7 was verified with the pinned requirements, and the installer handles RHEL 9 (no Python 3.7 package, SELinux, firewalld).

Release 1.6.5 answers the report that the GUI plots of the two real meters looked alike: every stage of the data path is keyed by the polled address (verified with emulated meters that differ in value, address and serial number), a meter that reports another address than the one polled is warned about, and `ab_check_meters.py` compares two real meters.

Release 1.7.0 replaces the GUI's picture previews by interactive plots (Section 9): one tab per meter, one plot per unit, a crosshair whose values are listed beside every plot, zoom and pan while the data update, and the meter's identity at the top of its tab.

2 Findings: RAM flush

2.1 Why the flush did not protect the GUI

The 1.5.5 flush logic only manages `TIME_SERIES_STORE`. In the GUI the dominant memory growth was elsewhere, so clearing the store could not help:

1. **Preview tabs leaked every poll.** `_populate_plot_tabs()` called `QTabWidget.clear()` and rebuilt every tab. Qt documents that `clear()` and `removeTab()` do *not* delete the page widgets [1]; each poll therefore left about 70 widgets plus their full-size pixmaps alive. Measured on the original code at a 2 s period: 1507 → 2947 → 4387 live widgets and 436 MiB → 691 MiB → 946 MiB RSS at 0 s, 45.8 s and 91.2 s (Fig. 6).
2. **The log console grew without bound** (`QTextEdit`, DEBUG level, one line per page fetch).
3. **The store-size trigger never cleared anything.** In headless mode `should_flush()` rewrote the files but left the store intact; the GUI never evaluated it at all. Only the system-RAM-% trigger cleared.
4. **The system-RAM-% trigger mis-fired.** It is system-wide: when other processes held the RAM, the monitor flushed and cleared its store on *every* poll without freeing anything

significant.

5. **Freed memory was not returned to the operating system.** `gc.collect()` frees objects, but the allocator keeps the pages, so RSS (and the RAM %) stayed high after a flush.
6. **FITS rewrites multiplied rows** (Section 4); with FITS enabled the GUI exhausted memory in minutes regardless of any flush.
7. **GUI-thread saturation.** About 40 preview PNGs were rendered on the GUI thread every poll and Poll Now ran the whole poll on it.

2.2 What 1.6.0 does

- Preview tabs are created once per plot and updated in place; tabs that disappear are `deleteLater()`'d. Only the visible tab is rendered, min/max decimated to `PREVIEW_MAX_POINTS` per series, and the pixmap of a tab is released when the user leaves it.
- The console is a `QPlainTextEdit` with `maximumBlockCount` [2], fed from any thread through a queued signal.
- Polling, persistence and flushing run in an acquisition worker thread (`QThread`); Save Now, Flush RAM Now and Open in Veusz are executed by that worker between polls, so they never race it.
- One flush function, `perform_ram_flush()`, serves the GUI, the headless loop and the caller. It persists every enabled output, waits for the writes, drops only rows that *every* output confirmed, and then returns the freed memory to the operating system: `malloc_trim(0)` on Linux [3], heap decommit and working-set trim on Windows since 1.6.1 (Section 2.4).
- Triggers are store estimate, process RSS, system RAM % and free RAM, with a minimum interval and a minimum store size for the system-wide triggers (Eq. 2).

2.3 Store estimate and trigger

For table i with n_i rows and k_i parameters, the store estimate in bytes is evaluated in $O(\text{tables})$ every poll:

$$B = \sum_i n_i \,(32\,k_i + 144), \tag{1}$$

32 B being one float object (24 B) plus its list slot (8 B) and 144 B per row the timestamp string, sequence number and Unix time. Against an exact object walk the estimate was within 1 % (5.54 MiB vs. 5.57 MiB; 22.16 MiB vs. 21.92 MiB). With Δt the time since the last flush, an automatic flush happens when

$$F = E \wedge (\Delta t \geq T_{\min}) \wedge \Big[\,(B > B_{\max}) \vee (B \geq B_{\min} \wedge (R > R_{\max} \vee P \geq P_{\max} \vee M_{\text{free}} < M_{\min}))\Big], \tag{2}$$

where E is `ENABLE_RAM_FLUSH`, $T_{\min}$ `MEM_FLUSH_MIN_INTERVAL_SEC`, $B_{\max}$ `MEM_FLUSH_THRESHOLD_MB`, $B_{\min}$ `MEM_MIN_STORE_MB_FOR_CLEAR`, R the process RSS, P the system RAM use and M_{free} the free RAM.

2.4 Returning memory to the operating system (1.6.1)

Dropping rows frees Python objects, but three layers below keep memory: CPython's `pymalloc` returns a 1 MiB arena (with `VirtualFree` or `munmap`) only when every object in it is free [19]; the C heap keeps freed blocks committed; and on Windows even released pages stay in the process's *working set* — Task Manager's "Memory" column — until the memory manager trims it. 1.6.0 handled only Linux, so on Windows a flush emptied the store but Task Manager showed no drop. With `MEM_RETURN_TO_OS = 1`, `return_memory_to_os()` now runs after every flush:

OS	Call	Effect
Linux (glibc)	`malloc_trim(0)`	free heap pages returned to the kernel [3]
Windows 8.1+	`HeapSetInformation(NULL, HeapOptimizeResources, {Version=1, Flags=0})`	every heap with a low-fragmentation heap: caches optimised, memory decommitted if possible [13]; the call Firefox makes under memory pressure [17]
Windows	`EmptyWorkingSet(GetCurrentProcess())` (`MEM_TRIM_WORKING_SET`)	removes as many pages as possible from the working set [14]; pages still in use fault back in on their next access
not used	`_heapmin()`	in the Universal CRT this is only `HeapCompact(__acrt_heap, 0)` [16], which "does not compact the heap any further" on a normal system [15]

The Windows functions are bound once through private `ctypes.WinDLL` instances with explicit `HANDLE` and `SIZE_T` argument types (the `ctypes` default C `int` would truncate 64-bit handles), the release is serialised by a non-blocking lock, runs outside the persistence lock and never raises into the poll loop. On Windows two numbers matter: the working set W (`psutil rss`, Task Manager "Memory") and the commit charge C (private bytes, `psutil vms`, Task Manager "Commit size"). With D the memory the heaps could decommit,

$$C_{\text{after}} = C_{\text{before}} - D, \qquad W_{\text{after}} \approx W_{\text{touched}} \le C_{\text{after}}, \tag{3}$$

where W_{touched} is the memory the process uses again before the next measurement. Both values are printed in the flush log and the GUI.

Measured with `tests/test_memory_release.py`: a store of 3 000 000 values (estimate 98.4 MiB) is filled and flushed.

	RSS (MiB)			
Run	idle	filled	flushed	OS step
Linux, Python 3.12.3	64	180	70	`malloc_trim`: 96 → 70
Linux, Python 3.8.20	51	169	54	`malloc_trim`: 80 → 54
Windows Python 3.12.15 (Wine)	350	466	364	both calls succeed*
GUI soak, Flush RAM Now (Linux)		239 → 220		`malloc_trim`

*Wine runs the Windows code paths but not the Windows memory manager; the Windows drop measured there comes from `pymalloc` freeing whole arenas.

3 Findings: iterative .vszh5 writing

3.1 How 1.5.5 wrote Veusz files

- It required the `veusz` Python package and drove `veusz.embed.Embedded`, which starts a Veusz process and rebuilds the *whole* document from the in-memory store on every save. Nothing was appended: the document could only reflect what was still in RAM.
- Consequently, every RAM flush had to produce a separate timestamped snapshot file, and the final file after a flushed run held only the last window. A long run became a scattered set of partial documents.
- Headless runs wrote Veusz only at the end of the run (and optionally at a flush); a crash lost the whole document. Caller mode rebuilt the full document, through a new Veusz process, on every poll.

- Dataset names were cut to 64 characters, so long parameter names could collide and silently overwrite each other.

3.2 How 1.6.0 writes them

`vszh5_writer.py` 0.0.18 adds `VeuszAppender`, which opens an existing document `'r+'` and grows it; the monitor uses it for one document per meter, `ABMeter_<ip>.vszh5`, appended on every poll:

- per table a date dataset `<octet>_dt_<table>` and one numeric dataset per parameter, collision-free and never truncated;
- datasets are chunked (1024 rows), resizable and *uncompressed*: a partly filled compressed chunk that grows is moved, leaving dead space, so compression would bloat a file appended every 30 s [4];
- the fill value is NaN, so rows reserved by a resize but never written (power loss mid-append) read back as gaps, not zeros; on the next append columns are padded or truncated back to the length of the time axis;
- a parameter that appears later is back-filled with NaN; the Veusz page script (per-table grid pages, per-unit overlay pages) is rewritten only when datasets are added;
- the table→dataset index lives in `/VszAppend`; Veusz 4.2.1 reads only `/Veusz` (confirmed in its `loadHDF5Doc` [5]), so appending resumes after a restart;
- when a reader holds the HDF5 file lock (Veusz while loading) the rows stay in memory for the next poll; if they must land now (flush, stop) a standalone rescue document is written; an unreadable file is moved aside and a new one started; rollover at `VSZH5_MAX_FILE_MB`.

Each append writes only the new rows: the cost per poll is O(new rows), independent of the document size. The appended documents load and render in real Veusz 4.2.1 (all 23 pages plus the PDF, Fig. 1).

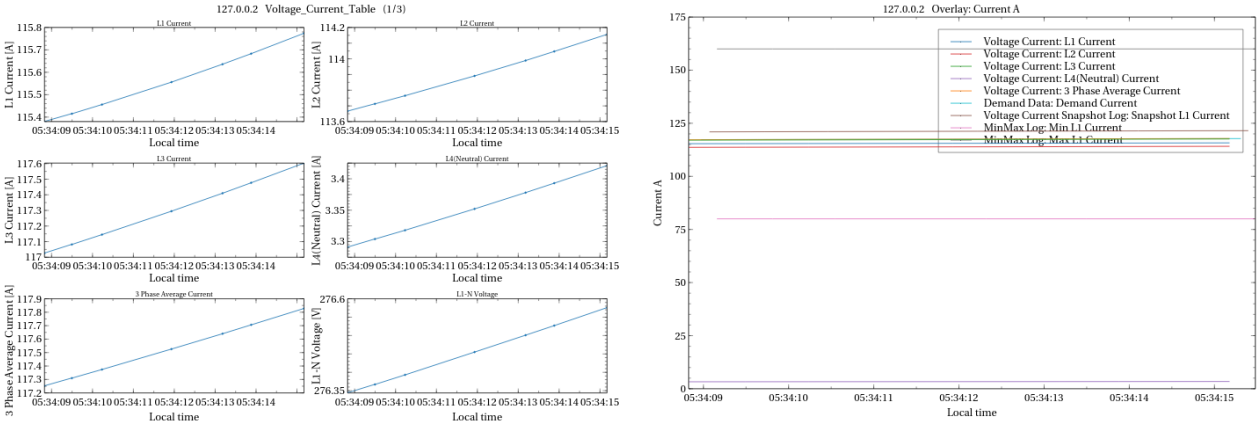


Figure 1: Pages of an appended `.vszh5` rendered by real Veusz 4.2.1: per-table grid page (left) and overlay page (right), emulator data.

4 Other defects fixed

FITS row multiplication

`EXTNAME = tname[:8]` made `Voltage_Current_Table`/`Voltage_Current_Snapshot_Log_Table` and `Diagnostic_Table`/`Diagnostic_Table_Extended` collide; the reader merged the two HDUs and each rewrite copied both tables' rows into both HDUs (Fig. 2). Same-second timestamp de-duplication also dropped genuine rows. 1.6.0 uses the full table name, separates old files by `TBLNAME`, adds `EPOCH_UTC`, and writes atomically (temporary file, then `os.replace`).

CSV / Markdown append
In append mode across runs, row counting mixed old and current rows and writing stopped after the first poll. Writers now keep per-row sequence cursors.

Exporter
Re-exported a dead meter's last values; `samples_total` plateaued (not a valid counter [6]). See Section 7.

Concurrency
`ALL_DEVICE_DATA` was rebuilt while the GUI iterated it; now swapped under a lock.

Duplicates
Duplicate IPs polled one meter from two threads; duplicate parameter names overwrote each other (now `Name [#N]`).

Python 3.12
`datetime.utcnow()` is deprecated [9, 10]; replaced by timezone-aware UTC.

Service wrapper
`POLL_INTERVAL` was read but never passed on.

Requirements
`qtpy` (imported by the GUI) was missing.

Duplicate rows after a busy writer (1.6.1)
1.6.0 advanced a writer's cursors in a future done-callback. `concurrent.futures` wakes the threads blocked in `result()` *before* it runs the done-callbacks [18], so a forced persist that waited for a busy writer could read the old cursor and send rows a second time; every writer appends or merges without de-duplication. Found as extra XLSX rows when the suite ran on Windows Python (slower writes); possible on any OS. Fixed in Section 5.2.

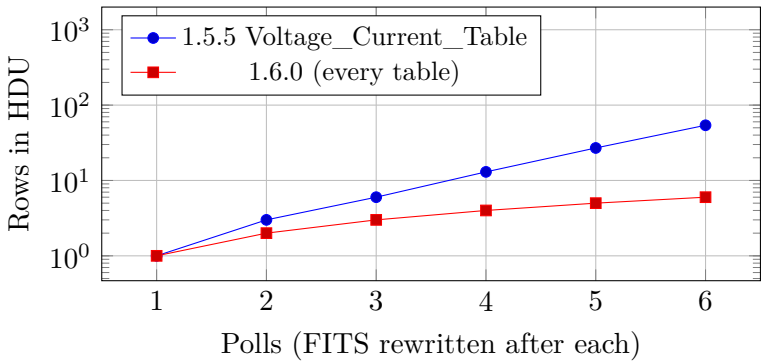


Figure 2: FITS rows per HDU after successive rewrites (emulator, measured).

5 Architecture

5.1 Threads

5.2 Persistence by sequence cursors

Each stored row r receives a process-wide increasing sequence number s_r. Every writer w keeps, per output folder and per (meter, table), the highest sequence number it persisted, c_w. A persist call copies once, under the store lock, the rows with $s_r > \min_w c_w$, and hands each writer the rows with $s_r > c_w$. A writer's cursor moves only after its write succeeded, and a writer whose previous write is still running is skipped (never two writes to one file). Since 1.6.1 the cursor is advanced inside the

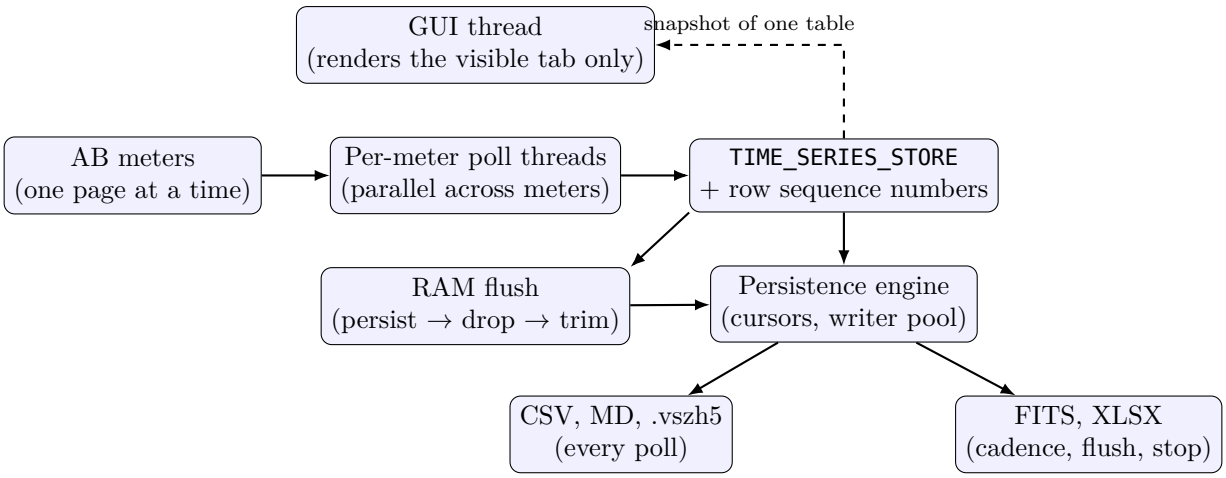


Figure 3: Data flow. The acquisition worker (GUI) or the headless loop runs the poll, the persistence and the flush in one thread; the GUI thread only reads snapshots of the table it displays.

writer task, before its future completes, and each writer's cursor is read only after its previous write is confirmed done at time t_{done}, so the rows handed to w are

$$R_w = \{\, r : s_r > c_w(t_{\text{done}}) \,\}, \tag{4}$$

which cannot contain a row already written (1.6.0 could read c_w before the callback that advanced it had run). A flush may drop row r only if

$$s_r \le \min_{w \in W} c_w, \tag{5}$$

with W the enabled outputs, which is the formal statement of "no row is dropped before every output has written it".

6 Windows operation (1.6.2, 1.6.3)

6.1 Output files held open by another program

Excel keeps an open CSV or workbook locked against writes by other programs. In Windows terms the file was opened with a share mode that lets others read but not write; a later open for writing fails with a sharing violation [21], which Python raises as `PermissionError` (Win32 errors 5, 32 or 33). 1.6.1 logged an ERROR for every table on every poll in that state, and a RAM flush could not drop the rows. 1.6.2 treats the file as *in use*:

- one WARNING per file per 10 minutes; the writer's cursor does not move, so the rows stay in memory and are written once the file is closed;
- a forced persist (flush, Save Now, stop) writes them to `<file>_<YYYYmmdd_HHMMSS>[_N]_rescue.csv` (or `.md`), so the flush can still free the memory;
- the next successful write first merges the rescue files, ordered by (time stamp, counter): each is renamed to `.merging` before its rows are copied, so a merge is never repeated even if deleting the file afterwards fails;
- new Markdown rows after a merged rescue section start a "(continued)" section.

FITS, XLSX and `.vszh5` already kept their rows or wrote rescue files.

6.2 Stopping with Ctrl+Break

On Windows `SIGTERM` sent by another process is `TerminateProcess` and cannot be handled [23]; supervisors send `CTRL_BREAK_EVENT` to the process group instead, which reaches only processes

sharing the sender's console [22]. The monitor (1.6.2) and the exporter (0.1.1) now handle `SIGBREAK` like `SIGINT`/`SIGTERM`: stop after the current page, then the final writes. Before, Ctrl+Break killed them without the final FITS/XLSX writes.

6.3 Windows test kit

`windows/` contains one-click scripts for a Windows 10/11 PC; no meters are needed (the emulator serves two meters on 127.0.0.2 and 127.0.0.3).

Script	What it does
`1_setup_environment.bat`	creates `env` from pinned conda-forge versions (mamba, conda or micromamba; venv + pip fallback); optional `env_veusz` with Veusz 4.2.1
`2_run_all_tests.bat`	`tests/run_all_tests.py`: system report, eight suites, `AB_TestResults_<computer>_<time>.zip`
`3_task_manager_check.bat`	six flush runs pausing 20 s each to watch the working set in Task Manager
`4_gui_with_emulator.bat`	GUI against the emulator, with a hands-on checklist
`5_headless_with_emulator.bat`	visible headless run, stopped with Ctrl+C or Ctrl+Break

The scripts use explicit `if errorlevel` checks instead of `||` after `call`, because `cmd` implementations disagree on the result of `||` after a called batch file (Wine's takes the branch even after success).

6.4 First results on real Windows (1.6.3)

The 1.6.2 kit ran on txrx03 (Windows 11 26H2, Intel Core i9, 32 GB) on 26–27 October 2026. Seven of eight suites passed: after a flush the working set fell from 298 to 4 MiB; a 10-minute headless soak wrote 591 polls as 591 rows in every output and stopped cleanly on Ctrl+Break.

Finding	Cause and fix
GUI closed with exit code −1066598273 (`0xC06D007F`) at the first preview plot	`0xC06D007F` is the Visual C++ delay-load "procedure not found" exception [25]. `faulthandler` [26] placed it inside `np.dot` (matplotlib's 3×3 transforms), i.e. in NumPy's BLAS: Intel oneMKL 2026.1.0, conda-forge's default on Windows. The headless monitor multiplies no matrices and was unaffected. The user's own environment with oneMKL 2026.2.0 worked; that build is labelled *broken* on conda-forge [27, 28] and cannot be pinned. With OpenBLAS (`libblas=*=*openblas`) the same GUI ran 80 polls writing every output without a warning; kit 1.1.0 pins it.
OpenBLAS: 2,971 MB commit charge at `import numpy`	per-thread work buffers. With `OPENBLAS_NUM_THREADS=1`: 18 MB (measured). The monitor sets it before NumPy is imported (`BLAS_MAX_THREADS`); a value set beforehand is kept.
Offline meter: about 67 s and 44 warnings per poll	each of 11 pages was retried. Once a page fails because the meter cannot be connected to at all (refused, no route, connect time-out), its remaining pages are skipped for that poll (`SKIP_PAGES_WHEN_UNREACHABLE`); an HTTP error from a page that did connect does not count.
"final writes: up to date" although no meter ever answered	now "no data received"; `run_status()` reports `polls_with_data`.
First poll in step 5 refused	the scripts waited a fixed 3 s for the emulator; they now wait until it answers (`tests/wait_for_port.py`, at most 30 s).

7 Prometheus exporter 0.2.0

The exporter keeps its own latest snapshot, `_LATEST[ip][table] = {fetch_epoch, values}`, replaced whole on a good fetch and removed on a failed fetch or when the meter is down; tables older than `--stale-after` (default 2.5× the interval) are not exported. Metric families are rebuilt on every scrape, as Prometheus recommends for exporters [6, 8]. Prometheus stores one sample per scrape; scraping faster than the poll interval records a reading more than once. Either match `scrape_interval` to `--interval`, or run with `--timestamps`, which attaches each reading's fetch time; Prometheus discourages exporter timestamps in general because they change staleness handling (series then persist for the 5-minute lookback) [7].

Metric	Meaning
`ab_meter_value{ip,table,parameter,unit}`	latest value
`ab_meter_up{ip}`	meter answered the last poll
`ab_meter_table_up{ip,table}`	table fetched and fresh
`ab_meter_last_fetch_timestamp_seconds`	last good fetch per table
`ab_meter_samples_total`	successful fetches (monotonic)
`ab_meter_table_fetch_errors_total`	failed fetches
`ab_meter_polls_total`	poll cycles
`ab_meter_poll_errors_total`	poll cycles that raised

8 Prometheus and Grafana (1.6.4)

8.1 Two ways to run the exporter

Mode	How the values reach Grafana
Continuous	poll every `--interval`; Prometheus pulls `/metrics` (job `ab_power_meter`), and/or every poll is pushed (`--push-gateway`, `--no-http`); `--count N` stops after N polls
Single run	`--once`: one poll pushed to a Pushgateway [29], exit code 0 (a meter answered), 1 (none), 2 (push failed); run by a systemd timer or the Windows Task Scheduler
Dict	`--print-json`: the dict as one JSON line per poll on stdout; from Python `poll_once()`, `iter_updates()`, `run(on_update=...)`

A push replaces the whole group `job/instance` (HTTP PUT) [30], so a meter that stops answering disappears from it and `ab_meter_up` becomes 0. Pushes carry no timestamps (the Pushgateway rejects them) and no process counters. The exporter starts none of the monitor's writers and no log file, and sets `sys.dont_write_bytecode`; a test compares the program and working folders before and after a single run.

8.2 Dashboard

`ab_grafana_dashboard.py` takes the meters' dict — from one headless poll, a running exporter, Prometheus' series API or a `--print-json` line — and writes the dashboard JSON (schema 36, Grafana 9 and later) [31]: an Overview row, then one row per meter with one time-series panel per unit. Every parameter of a meter with the same unit is a trace of the same plot, selected by `ab_meter_value` with the meter's `ip` and the panel's `unit` label, so a panel picks up all series of its unit by itself. Panels with more than six traces span the full width with the legend on the right (Grafana gives a bottom legend only about a third of the panel height); values without a unit form a table. In the repeat layout a single row is repeated for every value of the `ip` variable; Grafana regex-escapes multi-value variables for Prometheus, so those panels match the IP with a regular expression (`=~`). Provisioning files and an upload through the HTTP API [32, 36] are included.

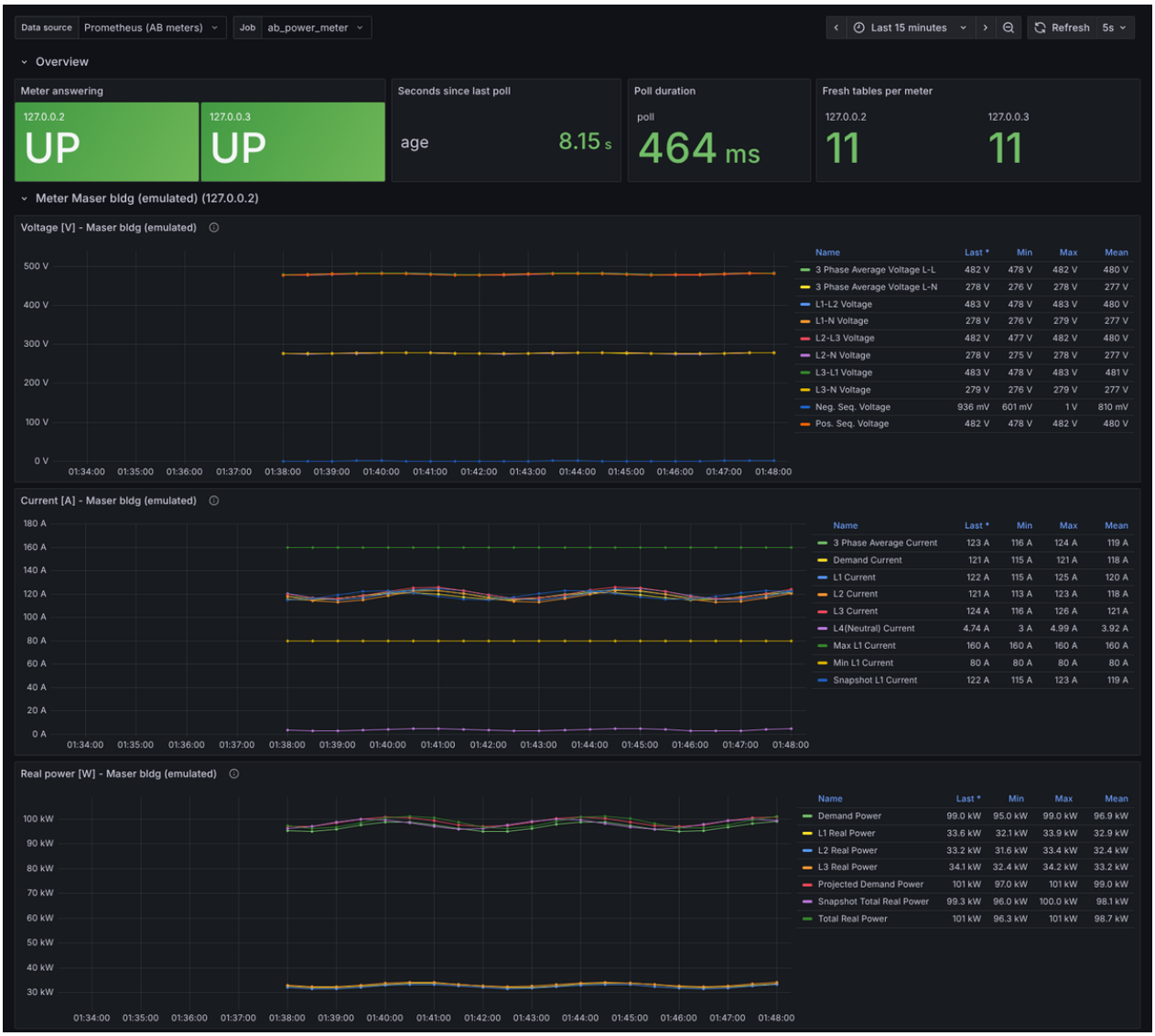


Figure 4: Per-meter dashboard in Grafana 12.2 (emulated meters, 5 s polls): Overview row, then the voltage, current and power panels of the first meter.

A Pushgateway keeps the last values until the next push, so *Meter answering* is computed as

$$u_{\text{shown}} = u \cdot [\, t_{\text{now}} - t_{\text{last poll}} < t_{\text{stale}} \,], \qquad t_{\text{stale}} = 3\,T, \tag{6}$$

with u the value of `ab_meter_up`, $[\cdot] = 1$ when true and 0 otherwise, and T the poll or schedule period; it shows DOWN / STALE once the runs stop.

8.3 Python 3.7 on RHEL 9

RHEL 9's `python3` is 3.9 and it has no Python 3.7 package, so the interpreter comes from conda-forge (3.7.12). `install.sh` 1.2.0 (`--mamba --py37`) creates it in `<prefix>/env` — 1.1.0 used a named environment in root's home, which the service account cannot read and SELinux does not let services execute — upgrades pip to 24.0 (the pip bundled with 3.7 cannot read `manylinux_2_28` wheel tags), installs wheels only, labels the environment `bin_t` when SELinux is enabled, can open the exporter port in firewalld (`--open-port`) and installs the single-run timer (`--once-timer`). `prometheus_client` 0.17.1 and pip 24.0 are the last releases for Python 3.7. The unit files now keep `StartLimitBurst` and `StartLimitIntervalSec` in `[Unit]`; systemd 252 (RHEL 9) ignores the interval in `[Service]`.

8.4 Verification

Emulated meters, the exporter on Python 3.7.12, Prometheus 3.5.0, Pushgateway 1.11.1 and Grafana 12.2.0 (official binaries, checksums verified): 274 continuous polls, 68 single runs (all exit code 0), and the per-meter, repeat and API-uploaded dashboards rendered in headless Chromium with data in every panel and no browser errors. All pins of `requirements-py37.txt` installed as wheels on Python 3.7.12 and the monitor, exporter, dashboard and memory suites passed there and on 3.12.3. `install.sh --mamba --py37 --once-timer` ran end to end with a stubbed `systemctl`; every unit passes `systemd-analyze verify`. Not run: a real RHEL 9 host, so the SELinux and firewalld branches are unexercised.

9 GUI

The window holds the output and memory-protection controls, the memory line, the bounded status console and the plots. The View menu toggles light and dark themes; the plots follow the theme. Closing the window waits for the final writes instead of terminating the worker thread.

9.1 Interactive plots (1.7.0)

Up to 1.6.5 the GUI showed one matplotlib picture per tab (one tab per table and per overlay group, about 40 tabs for two meters); reading a number meant opening a frozen copy in a separate window. Since 1.7.0 the plots are drawn by pyqtgraph [33] through the companion module `ab_live_plots.py` (Fig. 5):

- **One tab per meter.** At the top, the meter's identity line: polled address, the address the meter reports on its page 1 (red when it differs), serial number, rows in memory, and whether it answered the last poll.
- **One plot per unit** (V, A, W, VAR, VA, %, Hz, kWh, …) with every parameter of that unit as a trace — the grouping of the Grafana dashboard. The *View* box offers *by table* (one plot per table and unit) and *overlay groups*.
- **Crosshair and legend column.** The mouse over any plot places a crosshair in every plot of the meter (their time axes are linked). Beside each plot a table lists every trace with a tick box (hide or show), its value *at the cursor* (nearest sample; "-" inside a gap) and its *newest* value. The legend has its own column, so it never covers the newest data.

- **Zoom and pan while updating.** Curves are updated in place (`setData` [34]), so the view is kept. Left drag pans, right drag zooms one axis, Ctrl + wheel zooms; a plain wheel scrolls the tab (`LIVE_PLOT_WHEEL_NEEDS_CTRL`). Any manual zoom or pan switches *Follow newest* off, so a poll never moves the view from under the mouse; *Show* selects the follow window (10 min … 7 d, all). The right-click menu exports a plot as CSV, PNG or SVG.
- **Gaps.** A time step longer than `LIVE_PLOT_GAP_FACTOR` (3) times the median step is drawn as a gap instead of a line.

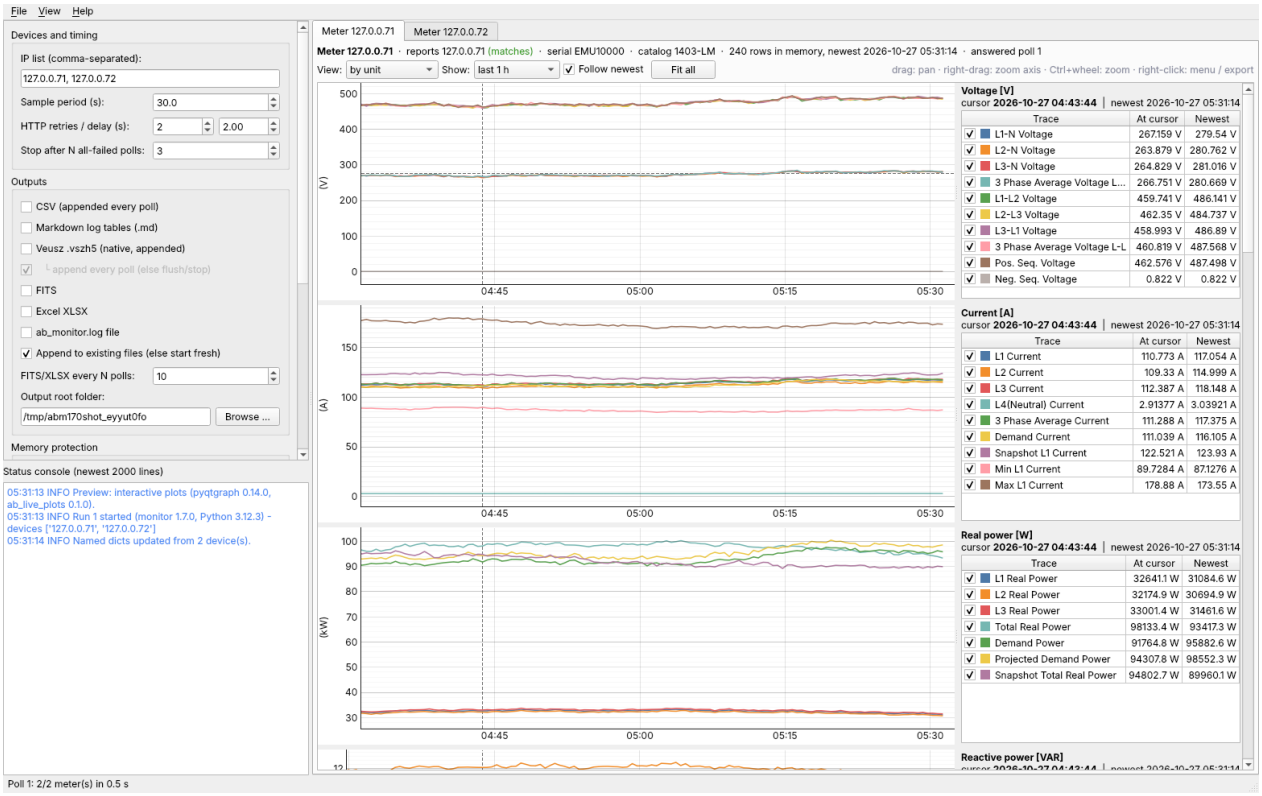


Figure 5: Monitor 1.7.0 GUI (off-screen capture): emulated meters with a synthetic two-hour history; cursor at 04:43:44, its values in the legend columns.

Data path and memory. The plots never touch the store directly: after a poll, only the *visible* meter tab calls `preview_groups(ip, view)`, which copies that meter's lists under the store lock, converts them to NumPy outside it and min/max-decimates each trace to `LIVE_PLOT_MAX_POINTS` (10 000; a one-sample spike survives); pyqtgraph thins further to the screen (peak downsampling, clip to view). Hidden tabs drop their arrays (`LIVE_PLOT_RELEASE_HIDDEN`) and are refilled when shown. Table 1 gives the cost for one emulated meter (44 traces).

Table 1: Cost of one meter's plots (`tests/live_plot_memory_check.py`; Python 3.12.3 / 3.7.12). "Arrays" is Python and NumPy memory held by the tab (tracemalloc); after hiding, 2.2 MB of pyqtgraph objects remain.

Rows per table	Store (MB)	Update after a poll (s)	Arrays (MB)	Hidden (MB)
1 000	3.4	0.03 / 0.02	2.7	2.2
10 000	33.7	0.06 / 0.05	6.4	2.2
50 000	168.6	0.30 / 0.25	9.0	2.2

Two leaks found and fixed. (1) pyqtgraph builds the right-click menus of every plot without a Qt parent; with PySide6 6.2 they outlived the deleted plot (11 per plot, 351 per change of view). They are now deleted with the plot. (2) Emptying a curve with `setData([], [])` leaves pyqtgraph's inner curve holding its last display arrays (views of the whole trace, 6.9 MB at 50 000 rows); a hidden tab now calls `clear()`.

Fallback. With `PREVIEW_BACKEND = "matplotlib"`, without pyqtgraph, or when pyqtgraph fails a start-up check (one plot is built and deleted: pyqtgraph 0.12.4 imports with PySide6 $\geq$ 6.3 but fails as soon as a plot is built), the GUI shows the 1.6 picture previews and logs a WARNING.

Python versions. pyqtgraph 0.14.0 needs Python $\geq$ 3.10, 0.13.3 is the last for 3.8 and 0.12.4 the last for 3.7 [35]; 0.12.4 works only with PySide6 $\leq$ 6.2.4, so `requirements-py37.txt` pins PySide6 6.2.4 (was 6.5.3). `tests/test_live_plots.py` passed on 3.12.3 (PySide6 6.12.0 and 6.11.2), 3.8.20 (PySide6 6.6.3.1) and 3.7.12 (PySide6 6.2.4).

10 Deployment

10.1 Environment (mamba)

```
mamba env create -f environment.yml        # Python 3.12 + pip deps
mamba activate abmeter
# or pure conda-forge:
mamba env create -f environment-conda.yml
# Python 3.7.11 hosts:
mamba env create -f environment-py37.yml
```

10.2 Services

```
sudo ./install.sh                    # headless monitor, system mode
sudo ./install.sh --exporter         # + Prometheus exporter
sudo ./install.sh --mamba            # mamba env, Python 3.12
python ab_stop.py                    # clean stop (STOP_COLLECTION)
```

The installer also installs `vszh5_writer.py`, `ab_live_plots.py` and `ab_check_meters.py` (1.2.1) and chooses `requirements-py312.txt` on Python $\geq$ 3.12. The wrapper passes `POLL_INTERVAL` and optional `RAM_PCT_LIMIT`; `ABMETER_OUTPUT_DIR` relocates the output.

10.3 Library use

```
import ab_power_meter_monitor as abm
cfg = abm.begin_run()                # folders, log file, run bookkeeping
status = abm.poll_cycle(cfg)         # poll + store + persist + memory check
abm.perform_ram_flush(cfg, "manual")  # persist, then free rows
abm.end_run(cfg)                     # final writes (FITS/XLSX included)
```

11 Configuration reference

Switch	Default	Meaning
`ENABLE_CSV`, `ENABLE_LOG_APPEND`	0	appended every poll
`ENABLE_VEUSZ`	0	native `.vszh5`, appended in place

Switch	Default	Meaning
`VEUSZ_APPEND_EVERY_CYCLE`	1	0: append at flush/stop only
`ENABLE_FITS`, `ENABLE_XLSX`	0	rewritten atomically on a cadence
`FULL_WRITE_EVERY_N_CYCLES`	10	FITS/XLSX cadence (0: flush/stop)
`APPEND_OUTPUT_FILES`	1	0: start files afresh once per run
`FSYNC_OUTPUTS`	1	fsync appended CSV/MD
`ENABLE_RAM_FLUSH`	1	automatic flush master switch
`MEM_FLUSH_THRESHOLD_MB`	256	$B_{\max}$ in Eq. 2
`MEM_PROC_RSS_LIMIT_MB`	0	$R_{\max}$ (0 = off)
`MEM_RAM_PCT_LIMIT`	60	$P_{\max}$
`MEM_FREE_MIN_MB`	512	$M_{\min}$
`MEM_MIN_STORE_MB_FOR_CLEAR`	1.0	$B_{\min}$
`MEM_FLUSH_MIN_INTERVAL_SEC`	120	$T_{\min}$
`RAM_FLUSH_KEEP_SAMPLES`	0	rows kept after a flush (GUI: 120)
`RAM_FLUSH_SPILL_PICKLE`	0	pickle the store when no output is on
`MEM_RETURN_TO_OS`	1	after a flush: `malloc_trim` / Windows heap decommit
`MEM_TRIM_WORKING_SET`	1	Windows: trim the working set after a flush
`VSZH5_MAX_FILE_MB`	256	`.vszh5` rollover
`VSZH5_TIME_AXIS`	local	`local` (DST-aware) or `utc`
`FITS_MAX_ROWS_PER_FILE`	100000	FITS rollover
`XLSX_MAX_ROWS_PER_FILE`	50000	XLSX rollover (Excel limit [12])
`METER_URL_TEMPLATE`	see right	`http://{ip}/{page}`; proxy / emulator
`SKIP_PAGES_WHEN_UNREACHABLE`	1	skip a meter's other pages for the poll once it cannot be connected to (1.6.3)
`BLAS_MAX_THREADS`	1	`OPENBLAS_NUM_THREADS` before NumPy loads, unless already set (1.6.3)
`PREVIEW_BACKEND`	pyqtgraph	`matplotlib`: 1.6 picture previews (1.7.0)
`LIVE_PLOT_MAX_POINTS`	10000	points per trace, min/max decimation (1.7.0)
`LIVE_PLOT_DEFAULT_VIEW`	unit	`table`, `overlay` (1.7.0)
`LIVE_PLOT_WINDOW_SEC`	3600	Follow-newest span; 0 = all (1.7.0)
`LIVE_PLOT_GAP_FACTOR`	3.0	step > factor × median step = gap; 0 = never (1.7.0)
`LIVE_PLOT_RELEASE_HIDDEN`	1	hidden meter tabs hold no plot arrays (1.7.0)
`LIVE_PLOT_WHEEL_NEEDS_CTRL`	1	0: the plain wheel zooms instead of scrolling (1.7.0)
`PREVIEW_MAX_POINTS`	2000	picture-preview decimation (fallback)
`GUI_LOG_MAX_LINES`	2000	console size

12 Verification

All tests run offline against `tests/ab_meter_emulator.py`, which serves the 11 pages per host with drifting values, string cells, a duplicated parameter name and optional faults. Python 3.12 runs used `-W error::DeprecationWarning`. The Windows runs used the conda-forge `win-64` build of Python 3.12.15 with numpy, h5py, astropy and openpyxl, executed under Wine 9.0 with Microsoft's own `ucrtbase.dll` [20].

Test	What it proves	Result
`test_vszh5_appender`	create/append/reopen, NaN back-fill, layout conversion, real-Veusz render	pass (3.12, 3.8, Windows 3.12)
`test_monitor_offline` (8 tests)	one row per poll in every output across a flush; append across runs; fresh mode; FITS separation; duplicate names; trigger gating; locked `.vszh5`; no row sent twice to a busy writer; 23 pages render in Veusz 4.2.1	pass (3.12, 3.8; Windows 3.12 three runs)
`test_memory_release`	bindings built once with 64-bit argument types; release never raises; flush of a 98 MiB store returns it (Linux RSS 180 → 70 MiB); switch off; no overlap	pass (3.12, 3.8, Windows 3.12)
`test_exporter_offline`	no duplicate series; dead meter not exported; monotonic counters; `--timestamps`; SIGTERM shutdown (POSIX)	pass (3.12, 3.8; Windows 3.12 collector tests)
`gui_soak_test` (150 s, 137–140 polls)	widget count constant (445; 447 with the 1.6.1 check boxes); previews rendered 137/137; no monitor errors; close waits for final writes	pass (3.12, 1.6.0 and 1.6.1)
`test_windows_file_locks`	files locked like Excel (`CreateFileW`, read-share): one warning per file, no error, rescue files on flush, merged back in order, no row lost or doubled	pass (3.12 Linux simulated; Windows 3.12 real locks)
`headless_soak_test`	real CLI, every output, automatic flushes, graceful stop; row counts equal in CSV, MD, `.vszh5`, XLSX, FITS; no leak	pass (3.12 Linux)
`test_ip_separation` (4 tests, 1.6.5)	pages of one meter strictly serial, meters in parallel; every output (store, CSV, MD, `.vszh5`, FITS, XLSX, preview, exporter) holds only its own meter's values and address; reported-IP mismatch warned	pass (3.12.3, 3.7.12)
`test_live_plots` (8 checks, 1.7.0)	every drawn value and every crosshair read-out equals the stored value of *that* meter; per-meter tabs; legend tick boxes; plain wheel scrolls, Ctrl+wheel zooms and stops following, a poll keeps a zoomed view; no widget left after view changes; hidden tab holds no arrays; both fallbacks	pass (3.12.3 with PySide6 6.12.0 and 6.11.2; 3.8.20; 3.7.12 with PySide6 6.2.4)
`gui_soak_test` 0.0.3 (pyqtgraph)	as above, plus data shown after every poll, crosshair every poll, a view change; fails if pyqtgraph is installed but the GUI fell back	pass: widget growth 0 (3.12, 3.7, 60–90 s; 3.12, 10 min, 549 polls)
`run_all_tests` 0.0.4 (Windows kit)	all suites + report + results zip	11/11 pass (1.7.0, quick, Linux: 3.12.3 and 3.7.12)
Headless profile (600 polls, all outputs, flush every 50)	Python-traced memory constant; RSS plateau (Fig. 7)	pass (3.12)

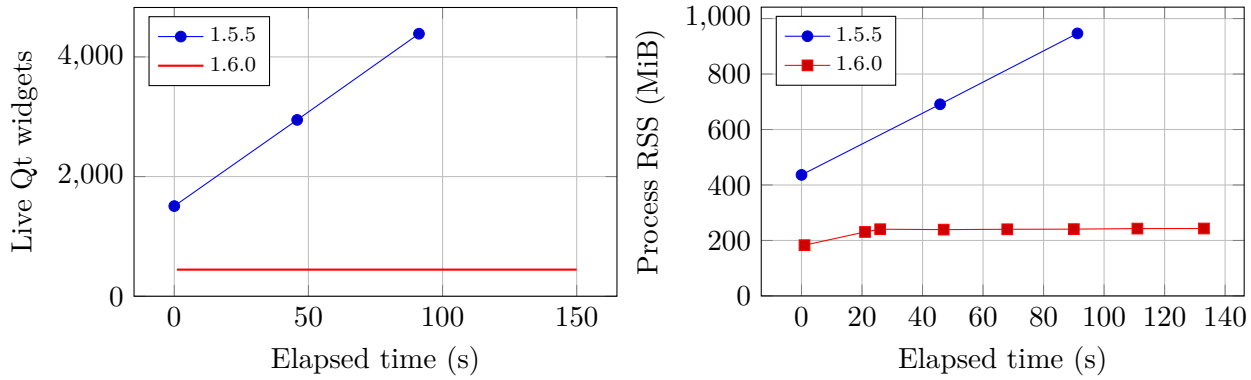


Figure 6: GUI soak, emulator, every output enabled. 1.5.5 at a 2 s period; 1.6.0 at a 1 s period with tab switching, an interactive window and a Flush RAM Now during the run (elapsed time of 1.6.0 points approximated from poll count, 140 polls in 150 s).

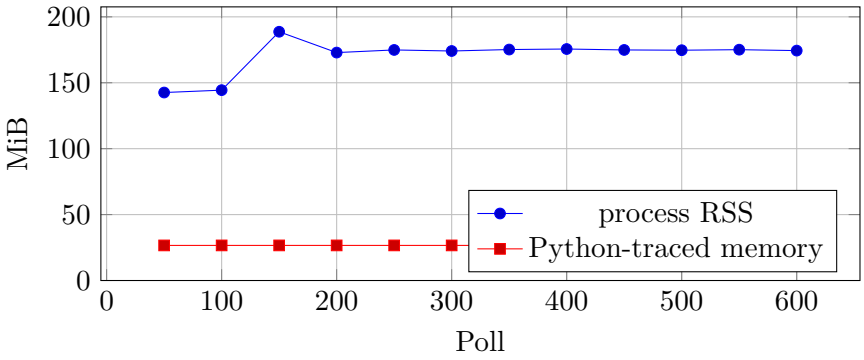


Figure 7: Headless monitor, 600 polls, CSV + Markdown + `.vszh5` every poll, FITS + XLSX every 25 polls, RAM flush every 50 polls (profiled with `tracemalloc`; the step at poll 150 is the profiler's own snapshot).

13 Module review

Module	Alternative	Decision
requests	httpx, urllib3	kept; one keep-alive session per meter; HTTP/2 or async bring nothing to a single-threaded meter web server
beautifulsoup4 + lxml	`lxml.html` directly	kept: BeautifulSoup already parses with libxml2 through lxml; direct lxml text extraction can differ for nested markup and would rename columns in existing files. Trees are now `decompose()`'d
matplotlib (Agg)	pyqtgraph	**replaced in 1.7.0** by pyqtgraph for the live plots (in-place `setData`, built-in downsampling, crosshair, no PNG round trip); matplotlib remains the fallback. fastplotlib (GPU) needs Python $\geq$ 3.10; Plotly needs a web engine
openpyxl	XlsxWriter	kept, now read-only $\rightarrow$ write-only streaming [11]; XlsxWriter is faster for pure writing but cannot read the existing workbook
astropy.io.fits	fitsio	kept (NRAO convention, no compiled extra)

Module	Alternative	Decision
veusz.embed	h5py + `vszh5_writer`	**replaced**: no Veusz process, in-place append, Python 3.12 wheels
psutil	/proc parsing	kept (cross-platform)
prometheus_client	—	kept; custom collector
qtpy + PySide6	Qt.py	kept (the project's Qt abstraction layer)

GPU acceleration. The workload is I/O bound (HTTP to embedded meters, small appends); the only numeric kernels are min/max decimation and array concatenation on at most tens of thousands of values per poll, which take microseconds in NumPy. A GPU path would add a dependency without a measurable benefit, so none was added.
Python 3.12. The interpreter's own improvements apply without code changes, e.g. inlined comprehensions (PEP 709) [10]; the code avoids the APIs deprecated in 3.12.

14 Known limitations and recommendations

- The Windows memory release was verified for binding, behaviour and error handling on Windows Python under Wine; the Task Manager drop itself follows from Microsoft's documented semantics and was not measured on a physical Windows machine for this release. Commit charge can stay above the working set when the heap is fragmented (it is re-used). On Windows 7 `HeapOptimizeResources` is not available (logged at DEBUG level); the working-set trim still works.
- The working-set trim makes pages still in use fault back in once (milliseconds). Setting `MEM_TRIM_WORKING_SET` to 0 leaves the working set to Windows.
- FITS and XLSX are rewrite formats; their cost grows with file size until rollover. Prefer CSV and `.vszh5` for long runs.
- FITS files written by 1.5.x with FITS enabled in the GUI may already contain mixed rows; start a new file.
- The local-time axis is ambiguous during the daylight-saving fall-back hour; `EPOCH_UTC` (FITS) and `VSZH5_TIME_AXIS = "utc"` avoid it.
- The matplotlib fallback's interactive window emits a matplotlib DeprecationWarning with PySide6 6.12 (third-party; harmless).
- pyqtgraph 0.12.4 (Python 3.7) labels its time axis with the UTC offset at start-up, so after a daylight-saving change the labels of older samples are one hour off until the GUI is restarted; 0.14.0 uses the offset of each tick. The legend values are not affected.
- A long GUI run grows the process by tens of MB before it levels off (10-minute soak, 549 polls: widgets constant, RSS flat after about 200 polls); the growth follows the store and the FITS/XLSX rewrites, and a RAM flush releases it.

References

[1] The Qt Company, "QTabWidget Class," *Qt 6 Documentation*. [Online]. Available: https://doc.qt.io/qt-6/qtabwidget.html (accessed Oct. 26, 2026).

[2] The Qt Company, "QPlainTextEdit Class," *Qt 6 Documentation*. [Online]. Available: https://doc.qt.io/qt-6/qplaintextedit.html (accessed Oct. 26, 2026).

[3] M. Kerrisk *et al.*, “malloc_trim(3),” *Linux man-pages*. [Online]. Available: https://man7.org/linux/man-pages/man3/malloc_trim.3.html (accessed Oct. 26, 2026).

[4] h5py developers, “Datasets,” *h5py documentation*. [Online]. Available: https://docs.h5py.org/en/stable/high/dataset.html (accessed Oct. 26, 2026).

[5] J. Sanders *et al.*, “Veusz 4.2.1 source code,” GitHub. [Online]. Available: https://github.com/veusz/veusz (accessed Oct. 26, 2026).

[6] Prometheus Authors, “Writing exporters,” *Prometheus documentation*. [Online]. Available: https://prometheus.io/docs/instrumenting/writing_exporters/ (accessed Oct. 26, 2026).

[7] Prometheus Authors, “Querying basics: Staleness,” *Prometheus documentation*. [Online]. Available: https://prometheus.io/docs/prometheus/latest/querying/basics/ (accessed Oct. 26, 2026).

[8] Prometheus Authors, “Custom Collectors,” *prometheus_client documentation*. [Online]. Available: https://prometheus.github.io/client_python/collector/custom/ (accessed Oct. 26, 2026).

[9] Python Software Foundation, “datetime — Basic date and time types,” *Python 3 documentation*. [Online]. Available: https://docs.python.org/3/library/datetime.html (accessed Oct. 26, 2026).

[10] Python Software Foundation, “What’s New In Python 3.12,” *Python 3 documentation*. [Online]. Available: https://docs.python.org/3/whatsnew/3.12.html (accessed Oct. 26, 2026).

[11] openpyxl developers, “Optimised Modes,” *openpyxl documentation*. [Online]. Available: https://openpyxl.readthedocs.io/en/stable/optimized.html (accessed Oct. 26, 2026).

[12] Microsoft, “Excel specifications and limits,” *Microsoft Support*. [Online]. Available: https://support.microsoft.com/en-us/excel/excel-specifications-and-limits (accessed Oct. 26, 2026).

[13] Microsoft, “HeapSetInformation function (heapapi.h),” *Microsoft Learn*. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/heapapi/nf-heapapi-heapsetinformation (accessed Oct. 26, 2026).

[14] Microsoft, “EmptyWorkingSet function (psapi.h),” *Microsoft Learn*. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/psapi/nf-psapi-emptyworkingset (accessed Oct. 26, 2026).

[15] Microsoft, “HeapCompact function (heapapi.h),” *Microsoft Learn*. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/heapapi/nf-heapapi-heapcompact (accessed Oct. 26, 2026).

[16] ReactOS project, “heapmin.cpp (Microsoft Universal CRT source),” *ReactOS Doxygen*. [Online]. Available: https://doxygen.reactos.org/d1/ddd/heapmin_8cpp_source.html (accessed Oct. 26, 2026).

[17] Mozilla, “Bug 1553296: Decommit pages in the Windows system heap upon memory-pressure event,” gecko-dev commit 0ef2cb2. [Online]. Available: https://source-hub.etos-pro.ru/mozilla/gecko-dev/commit/0ef2cb2e18803ecbcc6419f337e151144c06cefa (accessed Oct. 26, 2026).

[18] Python Software Foundation, “Lib/concurrent/futures/_base.py,” CPython 3.12 source. [Online]. Available: https://github.com/python/cpython/blob/3.12/Lib/concurrent/futures/_base.py (accessed Oct. 26, 2026).

[19] Python Software Foundation, "Objects/obmalloc.c," CPython 3.12 source. [Online]. Available: https://github.com/python/cpython/blob/3.12/Objects/obmalloc.c (accessed Oct. 26, 2026).

[20] WineHQ, "Wine 9.0," Ubuntu 24.04 package `wine64` 9.0~repack-4build3. [Online]. Available: https://www.winehq.org/ (accessed Oct. 26, 2026).

[21] Microsoft, "CreateFileW function (fileapi.h)," *Microsoft Learn*. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-createfilew (accessed Oct. 26, 2026).

[22] Microsoft, "GenerateConsoleCtrlEvent function," *Microsoft Learn*. [Online]. Available: https://learn.microsoft.com/en-us/windows/console/generateconsolectrlevent (accessed Oct. 26, 2026).

[23] Python Software Foundation, "subprocess — Subprocess management," *Python 3 documentation*. [Online]. Available: https://docs.python.org/3/library/subprocess.html (accessed Oct. 26, 2026).

[24] IAU FITS Working Group, "Definition of the Flexible Image Transport System (FITS), Version 4.0," 2018. [Online]. Available: https://fits.gsfc.nasa.gov/fits_standard.html (accessed Oct. 26, 2026).

[25] Microsoft, "Exceptions (C/C++) — delay load exception codes," *Microsoft Learn*. [Online]. Available: https://learn.microsoft.com/en-us/cpp/build/reference/exceptions-c-cpp (accessed Oct. 27, 2026).

[26] Python Software Foundation, "faulthandler — Dump the Python traceback," *Python 3 documentation*. [Online]. Available: https://docs.python.org/3/library/faulthandler.html (accessed Oct. 27, 2026).

[27] Anaconda, Inc., "conda-forge/mkl — Files," *anaconda.org*. [Online]. Available: https://anaconda.org/conda-forge/mkl/files (accessed Oct. 27, 2026).

[28] conda-forge, "Removing broken packages," *conda-forge documentation*. [Online]. Available: https://conda-forge.org/docs/how-to/emergencies/remove-broken-packages/ (accessed Oct. 27, 2026).

[29] Prometheus Authors, "When to use the Pushgateway," *Prometheus documentation*. [Online]. Available: https://prometheus.io/docs/practices/pushing/ (accessed Oct. 27, 2026).

[30] Prometheus Authors, "Exporting to a Pushgateway," *prometheus_client documentation*. [Online]. Available: https://prometheus.github.io/client_python/exporting/pushgateway/ (accessed Oct. 27, 2026).

[31] Grafana Labs, "JSON model," *Grafana documentation*. [Online]. Available: https://grafana.com/docs/grafana/latest/dashboards/build-dashboards/view-dashboard-json-model/ (accessed Oct. 27, 2026).

[32] Grafana Labs, "Provision Grafana," *Grafana documentation*. [Online]. Available: https://grafana.com/docs/grafana/latest/administration/provisioning/ (accessed Oct. 27, 2026).

[33] pyqtgraph developers, "pyqtgraph documentation," 2026. [Online]. Available: https://pyqtgraph.readthedocs.io/en/latest/ (accessed Oct. 27, 2026).

[34] pyqtgraph developers, "PlotDataItem," *pyqtgraph documentation*. [Online]. Available: https://pyqtgraph.readthedocs.io/en/latest/api_reference/graphicsItems/plotdataitem.html (accessed Oct. 27, 2026).

[35] pyqtgraph developers, "pyqtgraph," *PyPI* (release history and Python requirements). [Online]. Available: https://pypi.org/project/pyqtgraph/ (accessed Oct. 27, 2026).

[36] Grafana Labs, "Dashboard HTTP API," *Grafana documentation*. [Online]. Available: https://grafana.com/docs/grafana/latest/developers/http_api/dashboard/ (accessed Oct. 27, 2026).

A Distribution contents

Path	Content
`*.py` (repository root)	monitor, live plots (`ab_live_plots.py`), writer, exporter, dashboard builder, meter check, caller, examples, stop helper
`tests/`	meter emulator, offline, memory-release, file-lock, exporter, identity and live-plot tests, GUI and headless soaks, plot memory check, Windows memory check, test runner
`windows/`	Windows test kit 1.2.0 (scripts, pinned specs with OpenBLAS and pyqtgraph, `START_HERE`)
`monitoring/`	Prometheus configuration, Grafana provisioning and dashboard, systemd units (single-run timer, Prometheus, Pushgateway), Windows scripts, set-up guide
`docs/`	this manual (LuaLaTeX source and PDF), figures
`requirements*.txt`, `environment*.yml`	dependencies (3.12 primary, 3.8+, 3.7)
`install*.sh`, `*.service`, wrapper	deployment kit
`patch/`	`git format-patch` series against `main` (7a9800b)